

Proyecto final de Optimización Convexa

Optimización de hiperparámetros en modelos predictivos

Acoyani Garrido Sandoval

19 de mayo de 2026

1. Introducción

El objetivo del presente proyecto final es aplicar los conceptos aprendidos en el curso de Optimización Convexa para resolver un problema de optimización de hiperparámetros en un modelo predictivo.

En general, los pasos del proyecto son:

1. Seleccionar un problema de aprendizaje supervisado o no supervisado. Puede ser uno que esté siendo desarrollado, o que tenga relevancia personal o académica.
2. Elegir modelos base que tengan hiperparámetros susceptibles de ser optimizados.
3. Identificar los hiperparámetros críticos del modelo y definir un espacio de búsqueda adecuado para cada uno de ellos.
4. Plantear el problema de buscar los valores de los hiperparámetros como un problema de optimización: regularización L1, L2 o *elasticnet*, conjuntos convexos y restricciones, minimización de funciones convexas, dualidad y condiciones de optimalidad, o métodos de optimización tales como gradiente descendente.
5. Implementar la solución del problema en una Jupyter notebook. Está permitido el uso de librerías, siempre y cuando se justifique el método de optimización.

2. Selección de conjunto de datos

El conjunto de datos usado para estos fines es el **Telco Customer Churn** (Kaggle, 2018), un conjunto de datos desarrollado por IBM originalmente como una muestra de datos incluída con el software **IBM Cognos Analytics Server** (IBM, 2026), un software de análisis y visualización de datos basado en Java Enterprise Edition. Este conjunto de datos reviste una doble importancia personal para mí, ya que por una parte se trata de un conjunto de datos relacionado con comportamiento de clientes, giro en el cual actualmente manejo un negocio denominado **BuzónQR®**; y por otro lado, por el hecho de ser un conjunto de datos inicialmente concebido para probar las capacidades del software IBM Cognos Analytics Server, una plataforma de inteligencia de negocios inicialmente desarrollada por la empresa canadiense Cognos que fue comprada por IBM hacia el año 2008, orientada principalmente a visualización, procesamiento, análisis y exploración de datos; mi primer trabajo, el cual desempeñé en IBM desde 2012 hasta 2018, fue como administrador de una serie de instancias de dicha plataforma.

2.1. Características del conjunto de datos

El conjunto de datos *Telco Customer Churn* tiene como finalidad ofrecer un medio para mejorar la retención de clientes a través de información acerca de ellos y un dato que muestra si han dejado la empresa de telefonía en el último mes.

En este conjunto de datos, cada fila representa uno de los usuarios de la empresa de telefonía, y cada columna representa información acerca del cliente. Las características de este modelo son:

1. **Variable dependiente:** *Churn*. Es una variable binaria cuyos valores son “Yes” o “No”, e indica si el cliente ha dejado la compañía en el último mes. La distribución de clases de esta variable es de 73.5 % retenidos ($\Pr(Churn = No)$) y 26.5 % no retenidos ($\Pr(Churn = Yes)$); está moderadamente desbalanceado, lo cual es importante a la hora de elegir hiperparámetros de ponderación de clase.
2. **Características:** Encuadran en 4 grupos naturales: **demográficas** (*gender*, *SeniorCitizen*, *Partner*, *Dependents*), **información de cuenta** (*tenure* (meses como cliente), *Contract* (mensual, anual, bienal), *PaperlessBilling*, *PaymentMethod*), y **servicios suscritos** (*PhoneService*, *MultipleLines*, *InternetService*, *OnlineSecurity*, *OnlineBackup*, *DeviceProtection*, *TechSupport*, *StreamingTV*, *StreamingMovies*), y **cargos** (*MonthlyCharges*, *TotalCharges*).

Si realizamos un proceso de *one-hot encoding* de las características (descrito más adelante), obtenemos un total de 30 dimensiones de datos; lo cual implica que será necesario elegir características a través de la regularización, a la

vez que la cantidad de características es lo suficientemente pequeña para que un solver convexo lo pueda resolver.

En general, podemos observar que se trata de un problema de clasificación supervisada binaria, lo cual hace que el modelo natural a elegir sea **regresión logística**. Además de prestarse de forma natural a variables dependientes binarias, este modelo tiene 3 características que van en línea con el contenido del curso:

1. La regresión logística tiene una función de log-verosimilitud negativa convexa en parámetros, lo que hace que la superficie de pérdida tenga un mínimo global único.
2. Añadir regularización L1, L2 o elasticnet preservará la convexidad, lo que nos permitirá aplicarlas sin complejidades.
3. El problema de optimización tiene un planteamiento dual y condiciones KKT bien conocidas.

2.2. One-hot encoding de características

En la subsección anterior, mencioné que un proceso de *one-hot encoding* de las características arrojó 30 dimensiones de datos. El proceso para hacer eso y las implicaciones se explican a continuación.

¿Qué es? El *one-hot encoding* (codificación en caliente) es una técnica de preprocesamiento que convierte variables categóricas en variables numéricas binarias. Para cada variable categórica con k categorías distintas, se crean k columnas binarias (indicadoras), una por categoría, donde el valor 1 indica que la observación pertenece a esa categoría y 0 en caso contrario. En la práctica, para evitar multicolinealidad perfecta (*dummy variable trap*), se elimina una de las k columnas por variable, resultando en $k - 1$ columnas nuevas; la categoría eliminada queda implícitamente representada cuando todas las demás son 0.

Proceso El conjunto de datos *Telco Customer Churn* contiene 19 características (excluyendo *customerID* y la variable dependiente *Churn*). De ellas, 3 son ya numéricas continuas (*tenure*, *MonthlyCharges*, *TotalCharges*) y 1 es binaria numérica (*SeniorCitizen*), por lo que se mantienen sin transformación. Las 15 características categóricas restantes se procesan con `pd.get_dummies(..., drop_first=True)` de la librería *Pandas*, eliminando una categoría por variable para evitar la trampa de la variable ficticia. La tabla siguiente resume la expansión de dimensiones:

Característica	Categorías	Columnas generadas
<i>gender</i>	Female, Male	1
<i>Partner</i>	No, Yes	1
<i>Dependents</i>	No, Yes	1
<i>PhoneService</i>	No, Yes	1
<i>PaperlessBilling</i>	No, Yes	1
<i>MultipleLines</i>	No, No phone service, Yes	2
<i>InternetService</i>	DSL, Fiber optic, No	2
<i>OnlineSecurity</i>	No, No internet service, Yes	2
<i>OnlineBackup</i>	No, No internet service, Yes	2
<i>DeviceProtection</i>	No, No internet service, Yes	2
<i>TechSupport</i>	No, No internet service, Yes	2
<i>StreamingTV</i>	No, No internet service, Yes	2
<i>StreamingMovies</i>	No, No internet service, Yes	2
<i>Contract</i>	Month-to-month, One year, Two year	2
<i>PaymentMethod</i>	Bank transfer, Credit card,	3
	Electronic check, Mailed check	3
4 numéricas	(<i>SeniorCitizen</i> , <i>tenure</i> ,)	4
	(<i>MonthlyCharges</i> , <i>TotalCharges</i>)	4
Total		30

Implicaciones

1. **Dimensionalidad manejable.** Las 30 dimensiones resultantes son suficientemente pocas para que un solver convexo (como `liblinear` o `lbfgs` dentro de `sklearn.linear_model.LogisticRegression`) las maneje eficientemente.
2. **Necesidad de regularización.** Con 30 características y 7,043 observaciones, la razón de observaciones por característica es de aproximadamente 235:1, lo cual en principio es favorable. Sin embargo, varios subgrupos de variables indicadoras están correlacionados entre sí (p. ej., los servicios que dependen de *InternetService*), por lo que la regularización L1 o L2 sigue siendo conveniente para controlar la varianza del modelo y producir coeficientes interpretables.
3. **Interpretabilidad de coeficientes.** Cada coeficiente β_j en la regresión logística representa el cambio en el log-odds de *Churn* al pasar de la categoría de referencia a la categoría representada, manteniendo las demás características constantes; esto facilita la interpretación de resultados.
4. **Escala.** Las columnas numéricas continuas (*tenure*, *MonthlyCharges*, *TotalCharges*) tienen escalas distintas a las variables indicadoras $\{0, 1\}$, por lo que es necesario estandarizarlas (media cero, varianza unitaria) antes de ajustar el modelo con regularización, para que la penalización afecte a

todos los coeficientes de forma equitativa.

3. Desarrollo del problema

Habiendo expuesto la composición de nuestro conjunto de datos, la metodología que seguiremos será:

1. **Planteamiento del problema:** Predecir $\Pr(\text{Churn} = 1|x)$ a partir de las características de los clientes.
2. **Modelos:** Regresión logística como modelo primario, SVM lineal como modelo secundario. Compararé la pérdida logística contra la *hinge loss*. Privilegiaré el uso de modelos lineales con el fin de mantenernos en un marco de referencia convexo.
3. Plantear los hiperparámetros a optimizar y su espacio de búsqueda.
4. Plantear un problema de optimización convexa.
5. Implementación.

4. Modelo predictivo de regresión logística

4.1. Planteamiento del problema de optimización

4.1.1. Objetivo predictivo

Dado un vector de características $\mathbf{x} \in \mathbb{R}^{30}$ que describe a un cliente, queremos estimar:

$$\Pr(\text{Churn} = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \quad (1)$$

donde $\sigma(\cdot)$ es la función sigmoide, $\mathbf{w} \in \mathbb{R}^{30}$ es el vector de pesos que cada característica tiene en el resultado final, $\mathbf{x}$ es el vector de características leídas del conjunto de datos, y $b \in \mathbb{R}$ es el sesgo. La sigmoide es el fundamento de la **regresión logística**: si asumimos que el log-odds de la probabilidad es lineal en $\mathbf{x}$,

$$\log \frac{\Pr(\text{Churn} = 1 \mid \mathbf{x})}{\Pr(\text{Churn} = 0 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x} + b, \quad (2)$$

despejar $\Pr(\text{Churn} = 1 \mid \mathbf{x})$ da exactamente $\sigma(\mathbf{w}^\top \mathbf{x} + b)$. Cada componente w_j del vector $\mathbf{w}$ es la pendiente asociada a la j -ésima característica, análoga al coeficiente de una regresión lineal.

4.1.2. Función de pérdida (log-verosimilitud negativa)

Para un conjunto de entrenamiento $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ con $y_i \in \{0, 1\}$, la pérdida logística es:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \quad (3)$$

donde $\hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$. Esta función es **convexa en** $(\mathbf{w}, b)$, lo que garantiza la existencia de un mínimo global único.

4.1.3. Problema de optimización con regularización

Para controlar la varianza del modelo y forzar escasez o suavidad en los coeficientes, añadimos regularización:

$$\min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b) + \lambda \Omega(\mathbf{w}) \quad (4)$$

donde la penalización $\Omega(\mathbf{w})$ puede ser:

Regularización	$\Omega(\mathbf{w})$	Propiedad
L2 (Ridge)	$\frac{1}{2} \ \mathbf{w}\ _2^2$	Coefficientes pequeños, solución única
L1 (Lasso)	$\ \mathbf{w}\ _1$	Selección de características (<i>sparse</i>)
ElasticNet	$\alpha \ \mathbf{w}\ _1 + \frac{1-\alpha}{2} \ \mathbf{w}\ _2^2$	Combinación de ambas

Dado el desbalance de clases (73.5 % / 26.5 %), se añade también `class_weight='balanced'`, que pondera cada ejemplo i con $w_i \propto 1/\text{Pr}(y_i)$. El hiperparámetro principal a optimizar es $C = 1/\lambda$ (inverso de la intensidad de regularización): un C grande implica poca regularización; un C pequeño impone penalización fuerte y coeficientes más cercanos a cero.

4.2. Implementación

4.2.1. Preprocesamiento

Los pasos de preprocesamiento aplicados fueron:

1. Eliminar `customerID` (identificador sin valor predictivo).
2. Convertir `TotalCharges` a numérico; los espacios en blanco de clientes nuevos se imputan como 0.
3. Codificar `Churn` como variable binaria $\{0, 1\}$.
4. Aplicar *one-hot encoding* con `drop_first=True` a las variables categóricas.
5. Separar características (X) y variable dependiente (y).

6. Dividir en conjunto de entrenamiento (80 %) y prueba (20 %) con estratificación por clase.
7. Estandarizar las columnas numéricas continuas (*tenure*, *MonthlyCharges*, *TotalCharges*) con media 0 y varianza 1, ajustando el **StandardScaler** únicamente sobre el conjunto de entrenamiento para evitar fuga de información.

El resultado es un conjunto de entrenamiento de 5,634 observaciones y uno de prueba de 1,409, ambos con 30 características.

4.2.2. Modelo inicial y el solver L-BFGS

Se ajustó un primer modelo de regresión logística con regularización **L2**, $C = 1$ y `class_weight='balanced'`. El solver **lbfgs** (*Limited-memory Broyden-Fletcher-Goldfarb-Shanno*) minimiza internamente:

$$\min_{\mathbf{w}, b} \frac{1}{n} \sum_{i=1}^n [-y_i \log \sigma_i - (1 - y_i) \log(1 - \sigma_i)] + \frac{1}{2C} \|\mathbf{w}\|_2^2 \quad (5)$$

L-BFGS es un método de **quasi-Newton**: en lugar de calcular el Hessiano exacto $\nabla^2 \mathcal{L}$, lo aproxima iterativamente usando los últimos m pares $\{(\Delta \mathbf{w}_k, \Delta \nabla \mathcal{L}_k)\}$ para construir la dirección de descenso $\mathbf{d}_k = -H_k^{-1} \nabla \mathcal{L}(\mathbf{w}_k)$, seguida de *line search* para determinar el paso α_k :

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \mathbf{d}_k \quad (6)$$

Es el solver óptimo para este problema porque: (1) la convexidad estricta de la pérdida logística con L2 garantiza convergencia al mínimo global único; (2) la información de curvatura permite pasos más grandes que el gradiente descendente puro; (3) el costo de memoria $O(m \cdot p)$ es despreciable con $p = 30$; y (4) la regularización L2 es diferenciable en todo su dominio.

4.2.3. Evaluación del modelo base

Las métricas se calculan a partir de la matriz de confusión, que organiza las predicciones en cuatro celdas: Verdaderos Positivos (*VP*), Verdaderos Negativos (*VN*), Falsos Positivos (*FP*) y Falsos Negativos (*FN*). Las métricas son:

- **Precision** = $VP / (VP + FP)$: fracción de los predichos como “Abandona” que realmente abandonaron. Alta precisión implica pocas falsas alarmas.
- **Recall** = $VP / (VP + FN)$: fracción de los abandonos reales detectados por el modelo. Alto recall implica pocos abandonos sin detectar.
- **F1-score** = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$: media armónica entre ambas; útil con clases desbalanceadas.

- **Accuracy** = $(VP + VN)/n$: puede ser engañosa con desbalance (un clasificador que siempre predice “Retenido” tendría 73.5 % de accuracy).
- **Macro avg**: promedio simple entre clases, sin ponderar por tamaño; trata ambas clases como igualmente importantes.
- **Weighted avg**: promedio ponderado por el *support* de cada clase; refleja el desempeño global considerando el desbalance.

Para este problema, el **recall** de la clase positiva es la métrica de mayor peso de negocio: el costo de no detectar un cliente que abandona (falso negativo) supera al costo de una alarma innecesaria (falso positivo).

El **AUC-ROC** (*Area Under the Receiver Operating Characteristic Curve*) mide la capacidad discriminativa del modelo independientemente del umbral, graficando la tasa de verdaderos positivos contra la tasa de falsos positivos para cada umbral posible. Un valor de $AUC = x$ significa que, tomando al azar un par (cliente que abandona, cliente que no), el modelo asigna mayor probabilidad al primero en el $100x$ % de los casos. Un AUC de 0.5 equivale a un clasificador aleatorio; un AUC de 1.0 es un clasificador perfecto.

4.2.4. Validación cruzada del modelo base

Para obtener una estimación más robusta del desempeño se realizó una validación cruzada estratificada de 5 pliegues sobre el conjunto de entrenamiento. El procedimiento divide los datos en 5 subconjuntos manteniendo la proporción de clases; el modelo se entrena 5 veces usando 4 pliegues y evaluando en el restante. Esto reduce la varianza de la estimación y usa todos los datos para la evaluación. Los resultados fueron:

Métrica	Media	Desviación estándar
AUC-ROC	0.8454	± 0.0126
F1-score	0.6298	± 0.0220

Un AUC-ROC de 0.8454 indica que el modelo discrimina correctamente en el 84.5 % de los pares aleatorios. La desviación estándar baja en ambas métricas confirma que el desempeño es estable entre particiones. Estos valores constituyen la **línea base** contra la que se compararán los modelos optimizados; una mejora genuina deberá superar estos valores fuera del rango de ± 1 desviación estándar.

5. Modelo predictivo SVM

5.1. Planteamiento del problema de optimización

5.1.1. Del Perceptrón a la SVM

La **Máquina de Soporte Vectorial** (*Support Vector Machine*, SVM) es el sucesor conceptual del Perceptrón de Rosenblatt (1958). Ambos modelos buscan

un hiperplano que separe las dos clases en el espacio de características, pero con objetivos fundamentalmente distintos.

El Perceptrón aplica la regla de actualización

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i \quad \text{para cada ejemplo } i \text{ mal clasificado,} \quad (7)$$

donde $\eta > 0$ es la tasa de aprendizaje. Este proceso desciende por el gradiente de la pérdida cero-uno sobre el ejemplo mal clasificado, pero no tiene en cuenta la geometría global: puede producir un hiperplano con margen arbitrariamente pequeño, frágil ante perturbaciones en los datos de prueba. Si los datos no son linealmente separables el algoritmo directamente diverge.

La SVM corrige esto maximizando el **margen geométrico** $\rho = 2/\|\mathbf{w}\|$, que es la anchura de la franja libre de ejemplos a ambos lados del hiperplano. La diferencia de función de pérdida es clave: el Perceptrón usa la pérdida cero-uno $\ell = \mathbf{1}[y_i f_i \leq 0]$ (no convexa, no diferenciable), mientras que la SVM usa la *hinge loss* $\ell = \max(0, 1 - y_i f_i)$ (convexa, diferenciable casi en todo punto), lo que hace el problema de optimización tratable con herramientas de programación convexa.

Aspecto	Perceptrón	SVM
Objetivo	Cualquier hiperplano separador	Hiperplano de máximo margen (único)
Criterio de parada	Sin errores en entrenamiento	Optimalidad global (PQ convexa)
Datos no separables	Diverge	Resuelto con holgura ξ_i
Función de pérdida	Cero-uno (no convexa)	<i>Hinge loss</i> (convexa)
Generalización	Sin garantías teóricas	Cota VC minimizada

5.1.2. Problema de optimización: SVM de margen suave

Para datos no separables linealmente se introduce la **variable de holgura** $\xi_i \geq 0$, que mide cuánto viola el punto i el margen requerido. El problema de optimización primal es:

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n \xi_i \quad (8)$$

sujeto a:

$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i = 1, \dots, n \quad (9)$$

donde $\mathbf{w} \in \mathbb{R}^d$ es el vector normal al hiperplano (su norma determina el ancho del margen: $\rho = 2/\|\mathbf{w}\|$); $b \in \mathbb{R}$ es el sesgo; $C > 0$ controla el balance entre margen máximo y penalización de violaciones; y ξ_i vale 0 para puntos correctamente clasificados fuera del margen, $0 < \xi_i \leq 1$ para puntos dentro del margen pero correctamente clasificados, y $\xi_i > 1$ para puntos mal clasificados. El parámetro C tiene el mismo rol que en `LogisticRegression` de sklearn: $C = 1/\lambda$ es el inverso de la regularización.

5.1.3. El mapa de características $\phi(\mathbf{x})$ y el truco del kernel

El clasificador lineal en $\mathbb{R}^d$ solo puede aprender fronteras lineales. La solución es proyectar los datos a un espacio de mayor dimensión mediante el **mapa de características**:

$$\phi : \mathbb{R}^d \longrightarrow \mathcal{H} \quad (10)$$

donde $\mathcal{H}$ es el espacio transformado (posiblemente de dimensión infinita). La motivación teórica proviene del **teorema de Cover (1965)**: un problema de clasificación complejo proyectado a un espacio de alta dimensión tiene mayor probabilidad de ser linealmente separable.

Ejemplo explícito con kernel polinomial de grado 2 ($d = 2$). Dados $\mathbf{x} = (x_1, x_2)$:

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z})^2 = x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1 x_2 z_1 z_2 = \phi(\mathbf{x})^T \phi(\mathbf{z}) \quad (11)$$

con $\phi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1 x_2) \in \mathbb{R}^3$. El kernel calcula el producto interno en $\mathbb{R}^3$ *sin computar ϕ explícitamente*.

Kernel RBF (ϕ de dimensión infinita). El kernel gaussiano o de base radial es:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (12)$$

donde γ controla el alcance de la influencia de cada punto (γ grande $\rightarrow$ zonas de decisión locales; γ pequeño $\rightarrow$ fronteras suaves). Con `gamma='scale'`, sklearn usa $\gamma = 1/(d \cdot \text{Var}(X))$. Via la expansión en serie de Taylor de la exponencial se puede demostrar que ϕ construye *todas* las características polinomiales de todos los grados, por lo que $\mathcal{H}$ es de dimensión infinita: la SVM con kernel RBF tiene capacidad representativa ilimitada.

5.1.4. El espacio transformado: RKHS y teorema de Mercer

El espacio $\mathcal{H}$ es un **Espacio de Hilbert con Kernel Reprodutor** (*Reproducing Kernel Hilbert Space*, RKHS): un espacio vectorial de funciones dotado de un producto interno $\langle \cdot, \cdot \rangle_{\mathcal{H}}$ que satisface $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle_{\mathcal{H}}$.

El **teorema de Mercer** garantiza que toda función K simétrica ($K(\mathbf{x}, \mathbf{z}) = K(\mathbf{z}, \mathbf{x})$) y semidefinida positiva ($\sum_{i,j} c_i c_j K(\mathbf{x}_i, \mathbf{x}_j) \geq 0$) puede expresarse como:

$$K(\mathbf{x}, \mathbf{z}) = \sum_{k=1}^{\infty} \lambda_k \psi_k(\mathbf{x}) \psi_k(\mathbf{z}), \quad \lambda_k \geq 0, \quad (13)$$

confirmando que $\phi(\mathbf{x}) = (\sqrt{\lambda_1} \psi_1(\mathbf{x}), \sqrt{\lambda_2} \psi_2(\mathbf{x}), \dots)$. El kernel RBF y el kernel lineal son ambos semidefinidos positivos, por lo que el teorema aplica a los dos.

5.1.5. Derivación de la formulación dual

La formulación dual transforma el problema primal (en términos de $\mathbf{w}, b, \xi$) en un problema en términos de los **multiplicadores de Lagrange** α_i , uno por ejemplo. Esta transformación introduce el kernel de forma natural.

Paso 1 — Lagrangiano. Se introduce $\alpha_i \geq 0$ para la restricción de margen de cada punto i , y $\mu_i \geq 0$ para $\xi_i \geq 0$:

$$\mathcal{L} = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i \quad (14)$$

Paso 2 — Condiciones KKT de estacionariedad.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{0} \implies \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad (15)$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \implies \sum_{i=1}^n \alpha_i y_i = 0 \quad (16)$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \implies \mu_i = C - \alpha_i \implies 0 \leq \alpha_i \leq C \quad (17)$$

Paso 3 — Sustitución en el Lagrangiano. Usando (15) y (16):

- $\frac{1}{2} \|\mathbf{w}\|^2 = \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$
- $\sum_i \alpha_i y_i (\mathbf{w}^T \mathbf{x}_i) = \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$
- $b \sum_i \alpha_i y_i = 0$ (por (16))
- $\sum_i (C - \alpha_i - \mu_i) \xi_i = 0$ (por (17))

El Lagrangiano simplificado da el problema dual:

Paso 4 — Problema dual con kernel. Reemplazando $\mathbf{x}_i^T \mathbf{x}_j$ por $K(\mathbf{x}_i, \mathbf{x}_j)$:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \\ \text{s.a.} \quad & 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned} \quad (18)$$

El significado de cada elemento es:

Símbolo	Tipo	Significado
$\alpha_i \in [0, C]$	variable de opt.	Peso del punto i ; $\alpha_i > 0$ sii punto i es vector de soporte
$y_i \in \{-1, +1\}$	dato	Etiqueta de clase
$K(\mathbf{x}_i, \mathbf{x}_j)$	función	Similitud implícita en $\mathcal{H}$; reemplaza $\mathbf{x}_i^T \mathbf{x}_j$
$\sum_i \alpha_i$	término lineal	Favorece α_i grandes (mayor contribución al margen)
$\frac{1}{2} \sum_{ij} \alpha_i \alpha_j y_i y_j K_{ij}$	término cuadrático	Penaliza pares similares de la misma clase en conflicto
$\sum_i \alpha_i y_i = 0$	restricción lineal	Hiperplano balanceado entre clases

Una vez resuelto, la función de decisión para un nuevo punto $\mathbf{x}$ es:

$$f(\mathbf{x}) = \text{sgn} \left(\sum_{i=1}^n \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b \right), \quad (19)$$

donde la suma es efectivamente sobre los vectores de soporte ($\alpha_i > 0$) y b se obtiene de las condiciones de complementariedad KKT. La ventaja fundamental de la dualidad es que el problema dual tiene n variables (independiente de d), por lo que es factible usar kernels de dimensión infinita sin computar ϕ explícitamente.

5.1.6. Algoritmo SMO

`sklearn` resuelve el problema dual con el algoritmo **SMO** (*Sequential Minimal Optimization*): descompone el programa cuadrático en subproblemas de **dos variables** (α_i, α_j) a la vez que, dado que $\sum_k \alpha_k y_k = 0$, pueden resolverse analíticamente en forma cerrada. El algoritmo itera eligiendo el par que más viola las condiciones KKT hasta convergencia global.

5.2. Implementación

El preprocesamiento (one-hot encoding, división 80/20 estratificada, estandarización de columnas numéricas) es idéntico al de la regresión logística: se reutilizaron los mismos conjuntos `X_train`, `X_test`, `y_train` e `y_test` generados en la sección anterior, garantizando una comparación justa entre modelos.

5.2.1. Modelo inicial SVM (kernel RBF, $C = 1$)

Se ajustó un primer modelo SVM con kernel RBF, $C = 1$ (valor predeterminado de `sklearn`), `gamma='scale'` ($\gamma = 1/(d \cdot \text{Var}(X))$) y `class_weight='balanced'`, en las mismas condiciones que el modelo logístico base. Los resultados en el conjunto de prueba fueron:

Clase	Precision	Recall	F1-score	Support
Retenido	0.9025	0.7333	0.8092	1 035
Abandona	0.5141	0.7807	0.6200	374
Accuracy		0.7459		1 409

AUC-ROC: 0.8248. Vectores de soporte: 3 030 de 5 634 (53.8 % del conjunto de entrenamiento).

5.2.2. Evaluación del modelo base

Clase “Abandona” (clase de interés).

- **Recall = 0.7807:** El SVM detecta el 78.1 % de los clientes que realmente abandonan; deja sin detectar el 21.9 % restante. Es la métrica de mayor peso de negocio.
- **Precision = 0.5141:** De cada 10 alarmas de churn, algo más de 5 son correctas; las otras 5 son falsas alarmas. El valor bajo es consecuencia de `class_weight='balanced'`: el modelo sacrifica precisión para maximizar el recall.
- **F1 = 0.6200:** Balance razonable para una clase con 26.5 % de prevalencia.

Matriz de confusión.

	Predicho: Retenido	Predicho: Abandona
Real: Retenido	VN = 759	FP = 276
Real: Abandona	FN = 82	VP = 292

A umbral 0.5, el SVM genera 3.4 veces más falsas alarmas (276 FP) que abandonos no detectados (82 FN). Si el costo de un FN supera al de un FP —habitual en retención de clientes— reducir el umbral de decisión a ≈ 0.3 aumentaría el recall a costa de más FP, lo que puede ser más rentable.

Curva ROC. El AUC del SVM RBF (0.8248) es inferior al de la regresión logística (0.8417), con una diferencia de -0.0169 . La curva de la LR se sitúa por encima en la mayor parte del recorrido. Esto indica que con hiperparámetros predeterminados el SVM no supera a la LR en este dataset: las relaciones dominantes son en su mayoría **lineales** (tipo de contrato, servicio de internet, cargos), y el kernel RBF con $\gamma = \text{scale}$ modela la similitud de forma demasiado local. La optimización de C y γ es necesaria para aprovechar la capacidad no lineal del SVM.

Vectores de soporte (53.8 %). El alto porcentaje de vectores de soporte indica que el SVM está **sobrerregularizado** con $C = 1$: el margen es tan amplio que acepta muchas violaciones, usando más de la mitad del conjunto de entrenamiento para definir el hiperplano.

5.2.3. Interpretación de pesos: SVM lineal

El kernel RBF no produce coeficientes interpretables en el espacio original. Para obtenerlos se entrenó un SVM con kernel lineal ($K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$, $C = 1$,

`class_weight='balanced'`), que expone directamente el vector $\mathbf{w}$ del hiperplano de separación. Los 5 pesos de mayor magnitud absoluta fueron:

Característica	Peso w_j	Interpretación
<i>Contract_Two year</i>	-1.9996	Contrato bienal reduce fuertemente el riesgo de churn
<i>Contract_One year</i>	-1.9996	Contrato anual reduce fuertemente el riesgo de churn
<i>DeviceProtection_No internet service</i>	-0.2861	Sin internet (solo telefonía) → cliente estable
<i>OnlineBackup_No internet service</i>	-0.2861	Ídem
<i>InternetService_No</i>	-0.2861	Sin servicio de internet → menor riesgo de abandono

Todos los pesos de mayor magnitud son negativos, lo que indica que las variables más informativas son aquellas que **reducen** el riesgo de churn. El tipo de contrato domina con $|w_j| \approx 2.0$: los clientes en contratos de uno o dos años tienen compromisos que los retienen. Los clientes sin servicio de internet, que solo usan telefonía básica, también muestran menor propensión al abandono.

La comparación de F1 en el conjunto de prueba entre kernels fue: SVM lineal = 0.5876 vs SVM RBF = 0.6200. El kernel RBF mejora en 0.032 puntos de F1 al capturar interacciones no lineales entre las características, aunque a costa de interpretabilidad.

5.2.4. Validación cruzada

Se realizó la misma validación cruzada estratificada de 5 pliegues sobre el conjunto de entrenamiento, usando la misma partición `cv` aplicada a la regresión logística:

Métrica	SVM RBF	Regresión Logística	Diferencia
AUC-ROC	0.8283 ± 0.0103	0.8454 ± 0.0126	-0.0171
F1-score	0.6230 ± 0.0251	0.6298 ± 0.0220	-0.0068

Interpretación.

- **AUC-ROC (-0.0171):** La diferencia supera la desviación estándar del SVM (± 0.0103), por lo que no es atribuible a variabilidad aleatoria entre pliegues; es una señal real de que $C = 1$ y $\gamma = \text{scale}$ no son óptimos.
- **F1-score (-0.0068):** La brecha cae dentro de $\pm 1\sigma$ de ambos modelos, lo que indica que en la *detección de abandonos* los dos modelos son prácticamente equivalentes con parámetros predeterminados.
- **Estabilidad:** El SVM es marginalmente más estable en AUC (± 0.0103 vs ± 0.0126) pero más variable en F1 (± 0.0251 vs ± 0.0220), lo que sugiere que su frontera de decisión es más sensible a la composición de cada pliegue.

El diagnóstico conjunto —AUC inferior a LR tanto en prueba como en validación cruzada, más el 53.8 % de vectores de soporte— confirma que el SVM está

sobrerregularizado. Estos resultados constituyen la **línea base SVM** para la optimización de hiperparámetros: $\text{AUC-ROC} = 0.8283 \pm 0.0103$, $\text{F1} = 0.6230 \pm 0.0251$. Una mejora genuina deberá superar los umbrales $\text{AUC} > 0.8386$ y $\text{F1} > 0.6481$ (media $+1\sigma$).

Referencias

- IBM (2026). Ibm cognos analytics. <https://www.ibm.com/products/cognos-analytics>. Consultado en Kaggle el 16 de Mayo de 2026.
- Kaggle (2018). Ibm telco customer churn dataset. <https://www.kaggle.com/datasets/blastchar/telco-customer-churn/data>. Consultado en Kaggle el 16 de Mayo de 2026.